Hochschule Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

Web Architektur

Web Engineering 1 (319300)

Vorlesung 05

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-11-10

Agenda

- Layer-Modell
- Server Paradigmen
- Client Paradigmen
- Monolithen
- Microservices

Übersicht

Was ist (Web) Application Architektur

- Die High-Level Struktur einer Web Applikation
- Wie ihre Komponenten organisiert werden
- Wie sie miteinander interagieren
- Wie Verantwortlichkeiten zwischen Client, Server und Netzwerk verteilt sind

Drei Ebenen der Abstraktion

1. **System-Ebene**: Welche Komponenten existieren?
2. **Komponenten-Ebene:** Wie sind Verantwortlichkeiten verteilt?
3. **Code-Ebene:** Wie ist der Code intern strukturiert

Übersicht
— Definition

Mentales Modell:

- Architektur: Eine Menge an Entscheidungen, die man später nur schwer ändern kann
- Design: Füllt die Details innerhalb der Architektur

Übersicht
— Definition

Mentales Modell:

- Architektur: Eine Menge an Entscheidungen, die man später nur schwer ändern kann
- Design: Füllt die Details innerhalb der Architektur

Constraint der Web Umgebung: Alles passiert über den HTTP Request/ Response Zyklus zwischen Client und Server: Statelessness, Latenz und das Netzwerk sind unzuverlässige Medien, die Architekturentscheidungen beeinflussen

Übersicht
— Definition

Mentales Modell:

- Architektur: Eine Menge an Entscheidungen, die man später nur schwer ändern kann
- Design: Füllt die Details innerhalb der Architektur

Constraint der Web Umgebung: Alles passiert über den HTTP Request/ Response Zyklus zwischen Client und Server: Statelessness, Latenz und das Netzwerk sind unzuverlässige Medien, die Architekturentscheidungen beeinflussen

Grundfrage für jedes Paradigma: Wo soll die Logik der Anwendung leben und wie viel von dieser Logik soll im Browser laufen?

Übersicht
— Definition

Was entscheidet Architektur?

- **Wo wird die Seite gerendert?** Server, Client oder beides? (SSR, CSR, hybrid)
- **Data Flow & API Stil:** REST, GraphQL, gRPC, server push
- **State Location:** Server Session, Client Store, URL, Datenbank
- **Modularität:** Monolith, Modularer Monolith, Microservices, Micro-Frontends
- **Kommunikation:** Synchrone Requests/Responses, Asynchrone Events
- **Deployment:** Ein Artefakt, viele unabhängige Services

Übersicht
— Was entscheidet Architektur?

Warum ist es wichtig?

Konsequenzen von schlechter Architektur:

- **Schwer zu ändern:** Jedes neue Feature verlangt nun Änderungen an mehreren Orten
- **Schwer zu skalieren:** Ein aufwändiges Feature in einem Monolith kann das skalieren der ganzen App verlangen
- **Schwer zu testen:** UI, Logik und Datenbank sind alle in einem Codesegment
- **Schwierigkeiten im Team:** Mehrere Teams blockiert, weil sie auf der gleichen Codebase bzw. Pipeline arbeiten
- **Niedrige Performance:** falsche Rendering-Strategie (z.B. pures CSR für Seiten mit viel Inhalt)

Übersicht
— Warum ist es wichtig?

Warum ist es wichtig?

Konsequenzen von guter Architektur:

Qualitätsattribut	Welche Architektur erlaubt es?
Skalierbarkeit	Stateless Server, Horizontales Skalieren, Caching Tiers
Wartbarkeit	Separation von Aufgaben, Wenige interne Abhängigkeiten
Testbarkeit	Logik isoliert von I/O und UI
Deploybarkeit	Unabhängiges Deployen von einzelnen Komponenten
Performance	Richtige Rendering-Strategie für den Inhalt
Sicherheit	Trust Boundaries und Validierungsschichten

Übersicht
— Warum ist es wichtig?

Ökonomische Argumente:

- Architektur ist: Jetzt günstig, später teuer (ein Live-System umbauen kann 10-100x an Kosten verursachen)
- Conway's Law: Architektur wird auch das Team widerspiegeln → Entscheidungen für Architektur sind auch Teamweite Entscheidungen

Konkrete Beispiele:

- Netflix migrierte von Monolith zu Microservices
- Amazon: „You build it, you run it"
- Shopify: Bewusste Wahl eines modularen Monolithen

Beispiele

Netflix - Vom Monolith zu Microservices

Ausgangspunkt: Netflix begann 1997 als DVD-by-mail Geschäft und nutzte eine monolithische Java App mit Oracle DBs

Auslöser: 2008 ging der ganze Service down, weil Teile der Datenbank corrupted wurden. Es dauerte 3 Tage, bis er wieder online kam. Die monolithische Architektur wurde als *single-point-of-failure* erkannt. Neues Streamingbusiness verlangte außerdem Skalierungen, die monolithisch nicht möglich gewesen wären.

Beispiele
— Netflix - Vom Monolith zu Microservices

Die Migration

- Eine 7-jährige Reise zu hunderten Microservices auf AWS
- Monolith und Microservices liefen parallel für Jahre
- Heute: 700+ Microservices, die 15+ milliarden API Aufrufe, verteilt auf 190+ Länder, täglich verwalten
- Netflix baute eigene OSS Tools während dem Prozess:
  - Eureka (Service Discovery)
  - Hystrix (Circuit Breaker)
  - Zuul (Gateway)
  - Simian Army (Chaos Testing)

Layered Model

- Jedes spätere Paradigma baut auf dem Layered Model auf
- Entweder Variation, Anpassung oder Ablehnung des Modells

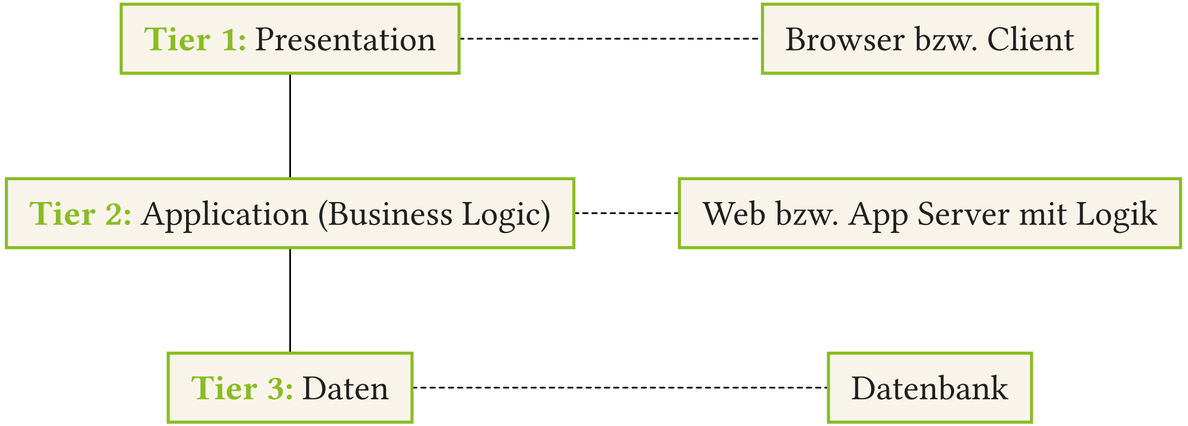
Kernidee: Layers werden genutzt, um Verantwortlichkeiten zu teilen, damit jeder Teil des Systems, gemessen an seinen Aufgaben, geändert werden und wachsen kann.

Aufgaben

Drei Aufgaben, die jede Application erfüllen muss:

1. **Darstellung:** Output rendern und User Interaktion verwalten (HTML, CSS, JS, …)
2. **Business Logic:** Die tatsächlichen Regeln der Domain (Ein Kunde kann eine Bestellung innerhalb von 30 Tagen)
3. **Datenzugriff:** State erhalten und behalten (SQL, ORM, Cache, externe APIs)

Klassisches 3-Tier-Model



Tier vs. Layer

Layers

- Logische Gruppen aus Code (Präsentation, Logik, Daten)
- Sie können alle in einem Prozess laufen

Tiers

- Physische deployment Einheiten
- Jeder Tier läuft auf seiner eigenen Maschine oder seinem eigenen Prozess
- Eine 3-Schichten App kann auf einem Tier oder drei deployed werden
- Geschichte des Aufbaus (1970er - 90er)
  - 1-tier (mainframe, alles in einer Box)
  - 2-tier (Client/Server, Clients interagieren direkt mit der Datenbank)
  - 3-tier (der heute Web-Standard)

Layered Model
— Tier vs. Layer

Tier vs. Layer

Die goldene(n) Regel(n):

- Die Präsentationsschicht darf die Business-Schicht aufrufen
- Die Business-Schicht darf die Daten-Schicht aufrufen
- Niemals in die andere Richtung
- Tiefere Schichten müssen nicht wissen, wer sie aufruft
- Business-Schicht sollte niemals SQL Strings enthalten
- Daten-Schicht sollte niemals HTML enthalten

Layered Model
— Tier vs. Layer

Beispiel

```java
@PostMapping("/checkout")
String checkout(@ModelAttribute CartForm form, Model model) {}
```

```java
public class CheckoutService {
  public Order placeOrder(Cart cart) {
    if (cart.isEmpty()) throw new EmptyCartException();
    if (!cart.customerHasSufficientCredit()) throw new
    PaymentException();
    return new Order(cart.items(), applyVat(cart.subtotal()));
  }
}
```

```java
public interface OrderRepository {
  void save(Order order);
  Optional<Order> findById(long id);
}
```

Server-Side Paradigms

Was würde passieren, wenn der Server (fast) alles selbst erledigt?

Server-Side Paradigms

Das Modell:

- Jede User-Interaktion löst eine volle HTTP-Request aus
- Der Server rendert eine komplett neue HTML Seite
- Der Browser ersetzt das Dokument
- Für 15 Jahre der Standard im Web
- Immer noch Standard bei Seiten mit viel Inhalt und SEO

Lebenszyklus einer Request

```
1 Click link → GET /products/42
2 → Router matches URL
3 → Controller receives request
4 → Model (business layer) loads/updates data
5 → View renders HTML from model data
6 → Response → browser paints full page
```


Durch den Zyklus sprechen:

- Am Beispiel bleiben: Klick auf Produkt in einer Übersicht → GET /products/42
- Router = Tabelle URL → Controller-Aktion; bei Rails/Django Konventionen (convention over configuration) statt manueller Konfiguration: GET /products/42 mappt automatisch auf ProductsController#show mit id=42
- Controller ist dünn: validieren, Model aufrufen, View auswählen, Logik gehört ins Model
- View ist auch dünn: nur Template + Model-Daten, keine Business-Logik (sonst wächst die Logik zurück in die View)
- HTTP ist zustandslos: der Server baut bei jeder Request den Kontext neu auf (Session-Cookie, ggf. Datenbank-Lookup), kein „eingeloggtes Benutzer-Objekt“ im Speicher, das zwischen Requests überlebt
- Historisch: 1995, 2010 Standard, klassisches LAMP/CGI, ab 2004 Rails machte MVC im Web populär (Django 2005, danach Spring MVC, Laravel)

Zwei Versionen von MVC

- **Original:** Controller erhält Input → Model update → Model benachrichtigt Observer → Views re-rendern
- **Web MVC (Rails, Django, Spring):**
  - Controller ist Request Handler
  - Ruft Model auf und schickt ein Template
  - Es gibt kein Observer-Pattern

Klassifizierung im Layered Model:

- View → Präsentationsschicht
- Model → Business-Schicht + Datenbank
- Controller → Verbindung zwischen Response und Request

→ MVC ist ein geschichtetes Modell mit Namenskonvention und Routing

Hintergrund: warum zwei Versionen?

- Original: Smalltalk-80, Trygve Reenskaug (1979), Desktop-GUI, keine Requests
- Observer-Pattern wirkt dort, weil View und Model dauerhaft im selben Prozess leben: Model ändert sich, View wird sofort benachrichtigt und neu gezeichnet
- Im Web gibt es dieses dauerhafte Verhältnis nicht: Request kommt, Antwort geht, Server „vergisst“ den Client (Request/Response ist one-shot)
- Daher: Web MVC ist nicht das klassische MVC, die Observer-Beziehung wird durch das Request/Response-Schema ersetzt
- In der Literatur heißt Web MVC auch „Model 2“ (JSP Model 2 Architektur); „Passive View“ für Views ohne eigene Logik
- Anschluss zum Kapitel Layered Model: MVC ist eine konkrete Ausprägung, Controller sitzt zwischen Transport- und Präsentationsschicht

Stärken:

- Einfaches mentales Modell
- SEO ist einfach da HTML die Response ist
- Kein Client JS Framework benötigt

Schwächen:

- Seiten neuladen fühlt sich träge an
- Jede Interaktion ist ein Round-Trip
- UI Features eher limitiert (SPA als Antwort)

Wie blieben MPAs am leben?

- AJAX für partielle Updates
- Turbolinks/Hotwire (Rails) und htmx als moderne MPA+ Ansätze

Definition: Ihr schreibt die Application-Logik als Funktionen (FaaS) oder nutzt bereitgestellte Backend Serivces (BaaS). Der Provider kümmert sich um Server, Skalierung usw.

Wichtig: Serverless heißt nicht *kein Server*. Nur das Server-Management wird abgegeben.

FaaS (Function-as-a-Service):

- AWS Lambda, Cloudflare Workers, Azure Functions, Google Cloud Functions
- Einheit im Deployment: Eine einzelne Function, die durch Event aufgerufen wird (HTTP Request)
- Skaliert mit Aufrufszahlen (wird berechnet nach Aufrufen)

Was hier wirklich passiert:

- Deployment-Einheit ist eine Funktion, nicht ein Prozess: kein eigenständiger Webserver, kein App-Server, der dauerhaft läuft
- Provider ruft die Funktion bei Bedarf auf („Event-getrieben“: HTTP Request, Datei-Upload, Cron, Queue-Nachricht)
- **Cold Starts:** Instanz wird bei Bedarf gestartet, der erste Aufruf ist langsamer (Lambdas: oft 100 ms, Sekunden je nach Runtime; Cloudflare Workers: V8-Isolates, Millisekunden)
- **Billing:** zahlt pro Aufruf + Ausführungszeit, bei null Traffic fallen Kosten gegen null (Achtung: API Gateway, Speicher etc. können trotzdem kosten)
- **Stateless by design:** Funktionen dürfen keinen dauerhaften lokalen State halten, State gehört in externe Stores (DB, Redis, S3)
- Typische Einsatzfälle: Webhooks, Cron-Jobs, Bildresizen nach Upload, kleine Glue-Logik, nicht: langlebige Verbindungen (WebSockets), CRON-freie Dauerprozesse

BaaS (Backend-as-a-Service):

- Firebase, Supabase, Auth0, Pocketbase
- Erfüllt häufige Backend-Anforderungen (Auth, Datenbank, File Storage, Push Notifications)
- Browser interagiert oft direkt mit dem BaaS
- Eigener Code wird genutzt um die Verbindung zu konfigurieren und Edge-Cases zu implementieren

Radikaler Punkt auf dieser Folie:

- Der Browser spricht direkt mit dem BaaS, kein eigener Server dazwischen! SDK im Frontend, z.B. `firebase.auth()` oder Supabase-JS-Client
- Fragt die Studierenden: wenn der Client direkten DB-Zugriff hat, was hindert einen Angreifer daran, beliebige Daten zu manipulieren?
- Antwort: Sicherheit liegt in den **Regeln**, nicht im versteckten Backend, Firebase Security Rules, Supabase Row Level Security (RLS) auf PostgreSQL-Ebene
- Das ist ein Paradigmenwechsel: Autorisierung wandert in die Datenschicht (deklarativ), statt in Controller-Code
- Trade-offs: schneller Start, wenig Boilerplate, aber Vendor Lock-in, begrenzte Query-Fähigkeiten, Regeln werden schnell komplex
- Einordnung der Angebote: Firebase/Supabase = alles (DB, Auth, Storage, Realtime); Auth0 = nur Auth; Pocketbase = self-hosted (ein Binary, eigene Kontrolle)

Thin Server vs. Thick Server

```
THIN CLIENT ←──────────────────────────────────→ THICK CLIENT
MPA, htmx,      Rails/Django      JSON API      SPA + BaaS,
SSR-first       full-stack apps    (server as    serverless glue,
"classic web"                      pure backend)  offline-first PWA
THICK SERVER ←─────────────────────────────────→ THIN SERVER
```

Thin Server vs. Thick Server

—

```
THIN CLIENT ←──────────────────────────────────→ THICK CLIENT
MPA, htmx,      Rails/Django      JSON API      SPA + BaaS,
SSR-first       full-stack apps    (server as    serverless glue,
"classic web"                      pure backend)  offline-first PWA
THICK SERVER ←──────────────────────────────────→ THIN SERVER
```

Thick Server / Thin Client:

- Der Server rendert, besitzt State, enthält ganze Logik
- Browser ist nur zur Darstellung da
- Pros: Logik an einem Ort, konsistentes Verhalten, wenige Anforderungen an den Client
- Cons: Jede Interaktion ist ein Round-Trip, Server Auslastung skaliert mit Zugriffen, nicht Usern

Thin Server vs. Thick Server

```
THIN CLIENT ←──────────────────────────────────→ THICK CLIENT
MPA, htmx,      Rails/Django      JSON API      SPA + BaaS,
SSR-first       full-stack apps    (server as    serverless glue,
"classic web"                      pure backend)  offline-first PWA
THICK SERVER ←─────────────────────────────────→ THIN SERVER
```

Thick Server / Thin Client:

- Der Server rendert, besitzt State, enthält ganze Logik
- Browser ist nur zur Darstellung da
- Pros: Logik an einem Ort, konsistentes Verhalten, wenige Anforderungen an den Client
- Cons: Jede Interaktion ist ein Round-Trip, Server Auslastung skaliert mit Zugriffen, nicht Usern

Thin Server / Thick Client:

- Server ist Daten/API Schicht, Browser hat State und Logik
- Grundprinzip der SPA

Thin Server vs. Thick Server

—

Thin Server vs. Thick Server

```
THIN CLIENT ←──────────────────────────────────→ THICK CLIENT
MPA, htmx,      Rails/Django      JSON API      SPA + BaaS,
SSR-first       full-stack apps    (server as    serverless glue,
"classic web"                      pure backend)  offline-first PWA
THICK SERVER ←─────────────────────────────────→ THIN SERVER
```

Thick Server / Thin Client:

- Der Server rendert, besitzt State, enthält ganze Logik
- Browser ist nur zur Darstellung da
- Pros: Logik an einem Ort, konsistentes Verhalten, wenige Anforderungen an den Client
- Cons: Jede Interaktion ist ein Round-Trip, Server Auslastung skaliert mit Zugriffen, nicht Usern

Thin Server / Thick Client:

- Server ist Daten/API Schicht, Browser hat State und Logik
- Grundprinzip der SPA

Wichtig: Das ist ein Spektrum und keine absolute Ausrichtung. Jede Anwendung richtet sich zwischen diesen beiden Ansätzen aus.

Thin Server vs. Thick Server

—

Vor dem ersten Pause-Schritt:

- Spektrum erst still wirken lassen; beim ersten Klick fragen: Rails und JSON API schicken beide Antworten vom selben Server, worin liegt der Unterschied? Nicht im Format, sondern darin, wo Logik und State liegen

Erklärung zu „skaliert mit Zugriffen, nicht Usern“:

- MPA: der Server rendert bei jedem Klick die komplette Seite, drei User, die viel klicken, erzeugen mehr Last als zehn ruhige User
- SPA: Rendering passiert im Client, der Server liefert nur Daten (billiger pro Interaktion)
- Trade-off: Rendering-Kosten werden auf die Geräte der User verlagert, alte Handys fühlen sich träge an, und auf euren Servern seht ihr davon nichts

Drei Leitfragen zur Einordnung einer Anwendung:

- Wo lebt der State? (Session/DB vs. Client Store)
- Wer rendert? (Server vs. Browser)
- Was passiert bei Verbindungsverlust? (MPA: gar nichts mehr; offline-first PWA: weiter arbeitbar)

Beispiele für Mischformen:

Vergleich

	MPA/MVC	Serverless/BaaS	SPA + API
Ort der Logik	Server, in Schichten	Provider + Funktionen	Client + API Server
Rendering	Server	Server oder Client	Client
State	Server Session / DB	Externe Stores	Client Store + Server
Ops Zuständigkeit	Ihr selbst	Provider	Ihr selbst
Typischer Stack	Rails, Django, Spring, Laravel	Lambda + API Gateway, Firebase, Supabase	React/Vue + REST/ GraphQL
Sweet Spot	Content Seiten, SEO, Forms	Kleine Teams, Prototypen	Interaktive Apps

Vergleich

—

Spaltenweise durchgehen, mit je einem konkreten Beispiel:

- MPA/MVC: Online-Shop oder Nachrichtenseite, SEO entscheidend, Crawler brauchen fertiges HTML
- Serverless/BaaS: Hackathon-Projekt oder MVP eines Startups, kleine Teams wollen keine Ops-Arbeit, brauchen aber sofort Auth und DB
- SPA + API: Dashboard, Editor, Mail-Client, hohe Interaktivität, immer eingeloggt, SEO irrelevant

Zwei Zeilen besonders betonen (Klausur-Kandidaten):

- „Ort der Logik“ und „Rendering“ sind die entscheidenden Fragen, aus ihnen folgt fast alles andere
- „Ops Zuständigkeit“: der eigentliche Verkaufspunkt von Serverless, bei den anderen Paradigmen pflegt ihr selbst Container, Updates und Zertifikate

Abschlussdiskussion (2, 3 Minuten):

- Welches Paradigma würdet ihr für das Semesterprojekt wählen? Begründung einfordern, die richtige Antwort verweist auf die Folie „Thin/Thick Server“: es ist ein Spektrum, keine Binärwahl

Client-Side-Paradigma

Was würde passieren, wenn der Client (fast) alles erledigt und der Server nur ein Dateninterface ist?

Client-Side-Paradigma

Wie kam das Paradigma zustande?

- Paradigma entstand ca. 2010 … warum?
- **JavaScript Engines:** V8 (2008) machte JS 10-100x schneller → Applikationen im Browser wurden möglich
- **AJAX:** Partielle Updates zeigten, dass User für eine ganze Session auf einer Seite bleiben
- **HTML5 APIs:** `history.pushState()`, `localStorage`, Application Cache, Canvas, Web Sockets
- **Smartphones:** Native Apps setzen den Maßstab für flüssiges Interagieren → User erwarten, dass Web Apps dasselbe Gefühl mitbringen
- **Gmail (2004) und Google Maps (2005)** waren der Proof-of-Concept

Client-Side-Paradigma
— Wie kam das Paradigma zustande?

Single-Page-Application (SPA)

Definition: Der Browser lädt eine HTML-Shell und ein JavaScript-Bundle. Jede weitere Navigation passiert auf der gleichen Seite. Der Server liefert Daten, keine Dokumente.

Neues Konzept: **Client-Side-Routing**

- Die URL ist State auf der Userseite
- URL wird vom JS-Router interpretiert
- Bei URL-Wechsel:
  - Keine Anfrage an den Server
  - Router überprüft den Pfad und wechselt die View-Komponente
  - `history.pushState()`, damit die URL teilbar bleibt

Probleme:

- Back/Forward im Browser muss berücksichtigt werden
- Beim Neuladen muss die Seite basierend auf der URL rekonstruiert werden

Single-Page-Application (SPA)
—

```
1   Load shell.html + app.js
2   → Router matches URL → renders view components
3   → View fetches data from API (JSON)
4   → Render → user interacts → router swaps views (repeat)
```

Neues Problem: Client State

- State in einer SPA lebt an mehreren Orten gleichzeitig (Komponenten, Client-Side-Store (Redux), URL und `localStorage`)
- Keine festgelegte Parität zwischen den States

3 Ansätze:

- Server-Cache-State: Daten leben auf dem Server (z.B. TanStack Query)
- Client-Side-Cache: State wird in der UI gespeichert
- URL-State: State über Filter und Pagination

Stärken:

- Interaktivität wie bei einer App
- Keine vollen Reloads
- Viele Möglichkeiten für komplexe UIs
- Klarer API-Vertrag kann mehrere Clients ermöglichen

Schwächen:

- **Time-to-First-Byte vs. Time-to-Interactive:** HTML kommt schnell an, danach JS downloaden, parsen, ausführen, Daten fetchen, rendern: Kann bei langsamen Verbindungen lange dauern
- **JS Bundle Wachstum:** JS bringt mehr Kosten pro Byte als HTML
- **SEO Probleme:** Crawlers können JS ausführen aber HTML vom Server ist
- **Das Rad neu erfinden:** SPAs implementieren viele Browsernative Funktionen neu (Routing, Scroll Restoration, ...)

Progressive Web Apps (PWA)

Letzter Schritt beim Thick-Client: Der Client kann komplett ohne den Server laufen (zumindest temporär)

Drei Prinzipien:

1. **Service Worker:** Ein programmierbarer Proxy zwischen Seite und Netzwerk. Fängt Requests ab und stellt Daten vom Cache bereit → Offline Funktionalitäten
2. **Web App Manifest:** Metadaten (Name, Icons, Display Mode) erlauben „Installieren“ der App zum Home Screen
3. **HTTPS:** Benötigt für die Service Worker mit eingebauter Sicherheit

Progressive Web Apps (PWA)

Vergleich

	MPA	SPA	PWA
Erstes Rendern	Server, schnell	Client, langsam (JS + Daten)	Client, langsam, danach sofort
Navigation	Volles Laden der Seite	Wechseln der Routen in der Seite	In der Seite, Offline
SEO	Sehr gut	benötigt Workaround	benötigt Workaround
State	Server Session	Aufteilung Client + Server	Zuerst lokal
Funktioniert offline	Nein	Nein	Ja
Komplexität	gering	mittel-hoch	hoch

,

Vergleich
—

Server-Side Rendering (SSR) & Hydration

Die Idee: Isomorphes Rendering

- Gleicher Code rendert auf dem Server **und** im Browser
- Ein Paradigma, zwei Laufzeitumgebungen
- Antwort auf die SPA-Schwäche: langsames erstes Rendern

Server-Side Rendering (SSR) & Hydration
— Übersicht

Die Idee: Isomorphes Rendering

- Gleicher Code rendert auf dem Server **und** im Browser
- Ein Paradigma, zwei Laufzeitumgebungen
- Antwort auf die SPA-Schwäche: langsames erstes Rendern

Lebenszyklus:

1. `GET /products/42`
2. Server führt die Komponenten aus → generiert HTML
3. Browser zeigt sofort das fertige HTML (schnelleres erstes Darstellen und SEO)
4. JS-Bundle lädt und führt *Hydration* aus: Framework hängt Event-Listener and DOM
5. Seite wird interaktiv

Server-Side Rendering (SSR) & Hydration
— Übersicht

Was passiert hier wirklich?

- Der Browser muss den kompletten Komponentenbaum erneut ausführen, um Event-Listener anzuhängen
- Das HTML ist lange da, aber *tot* bis Hydration fertig ist
- Nutzer sieht eine volle Seite, bevor sie interaktiv ist

Zwei Zeitpunkte driften auseinander:

- Time-to-First-Paint: schnell (Server liefert HTML)
- Time-to-Interactive: langsam (JS laden, parsen, ausführen, Hydration)

Folge:

- Meistens ist der größte Teil einer Seite statisch
- Hydration lässt ganze Seite für wenig Interaktivität bezahlen

Server-Side Rendering (SSR) & Hydration
— Der Preis

- **Gegenidee:** Nicht die ganze Seite hydraten
- Nur interaktive Teile werden hydrated
- Seite besteht aus statischem Server-HTML
- Nur einzelne *Inseln* werden hydriert
- Alles außerhalb der Inseln liefert null JavaScript

- **Gegenidee:** Nicht die ganze Seite hydraten
- Nur interaktive Teile werden hydrated
- Seite besteht aus statischem Server-HTML
- Nur einzelne *Inseln* werden hydriert
- Alles außerhalb der Inseln liefert null JavaScript

Entscheidung fällt pro Komponente, nicht pro App

- `client:load` → sofort hydrieren
- `client:idle` → wenn der Browser nichts zu tun hat
- `client:visible` → erst wenn die Insel sichtbar ist

- **Gegenidee:** Nicht die ganze Seite hydraten
- Nur interaktive Teile werden hydrated
- Seite besteht aus statischem Server-HTML
- Nur einzelne *Inseln* werden hydriert
- Alles außerhalb der Inseln liefert null JavaScript

Entscheidung fällt pro Komponente, nicht pro App

- `client:load` → sofort hydrieren
- `client:idle` → wenn der Browser nichts zu tun hat
- `client:visible` → erst wenn die Insel sichtbar ist

Beispiel: Astro (framework-agnostisch)

	SSR + Hydration	Islands
JS für statischen Content	Ja (Framework)	Null
Granularität	Ganze Seite	Einzelne Komponenten
Sweet Spot	Interaktive Apps	Content-Seiten

Monolith

Ein System, dessen kompletter Funktionsumfang in einer Codebase entwickelt wurde, als ein Artefakt gebaut wird und als eine Einheit deployed wird.

Modularität wird intern festgelegt. Verteilung ist extern

	Intern: verzweigt	Intern: modular
Extern: einzelnes Deployment	Großer Haufen an Software	Modularer Monolith
Extern: verteilt	verteilter Monolith (das schlimmste)	Microservices

Stärken und Schwächen

Stärken

- Eine Build/Deploy/Test Pipeline
- Wenige Netzwerkanfragen
- Einfaches Refactoring über Module hinaus
- Einfaches lokales Entwickeln

Schwächen

- Skalierung ist *alles oder nichts* (ein großes neues Feature skaliert ganzes System)
- Ein Crash zerlegt das ganze System
- Teams können sich an einer Codebase in die Quere kommen

Monolithen sind in den meisten Fällen die richtige Wahl.

Monolith
— Stärken und Schwächen

Microservices

Definition: Ein Architekturstil, der eine Applikation in unabhängige Services unterteilt. Jeder Service hat eine eigenen Daten, die um Business Anforderungen gebaut sind.

Wichtiges Axiom

- Microservices leben entlang der Domain Grenze, keiner technischen Grenze
- **Gut:** Order Service, Payment Service, Inventory Service
- **Schlecht:** UI Service, Logic Service, Database Service

Microservices

- **Autonomie für Organisation:** Teams können Services end-to-end besitzen und deployen
- **Unabhängiges Skalieren:** Netflix: Die Recommendation-Engine kann zu hunderten Instanzen skalieren, aber der Invoice-Service läuft auf zwei.
- **Fehlerisolierung:** Ein fehlerhafter Service bringt nicht die ganze Anwendung zum anhalten (sofern Implementation stimmt)
- **Technologische Optimierung:** Jeder Service erhält die besten Technologien für seine Anforderungen
- **Risikoverringerung beim Deployment:** kleine, häufige, rücksetzbare Deploys

Die Kosten

- **Netzwerk ersetzt Methodenaufrufe:** Latenz, Retries, Timeouts… jeder Aufruf zwischen Services kann auf Wegen fehlschlagen, die bei Funktionsaufrufen nicht passieren würden
- **Verteilte Transaktionen:** *siehe ACID*
- **Mehr Ops Aufwand:** 50 Services brauchen Discovery, Config, Observability und ein Team auf sie zu bauen → viel Overhead
- **Datenparität:** Service-Datenbanken bringen Composition for Anfragen auf andere Services

Wichtige Regel: Services dürfen andere Service-APIs Anfragen und Subscriben, aber niemals die Datenbank dieser Services direkt ansteuern.

Microservices
— Die Kosten

Auswahlverfahren

Welche Architektur wählt man nun?

1. Starte mit einem modularen Monolithen, außer es gibt spezifische Anforderungen

> **Fowlers Regel:** Man kennt selten die vollen Grenzen aller Services früh in der Entwicklung. Falsch gesetzte Grenzen sind mit die teuersten Fehler, die man später ausbessern muss.

2. Extrahiere einen Service, wenn das Modul tatsächlich anders ist (z.B. Skalierungsprofil, Verfügbarkeitsanforderungen, Frequenz des Deployments...)
3. Extrahiere niemals wegen Technologischen Anforderungen
4. Achte auf Signale, dass ein Wechsel gut wäre: Komplizierte Deployments, Team Koordination wird schwer...

Auswahlverfahren
—